

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (Bsc.IT Semester 3)
DSA

Practical 5

Roll No. : 45	Name: Raj Upadhyay
Class : SYBSC IT	Batch : 1st
Date of Assignment : 22/07/2026	Date/Time of Submission : 22/07/2026

Practical 5A

Binary Search Tree: Write a program to:

a. Create a binary search tree.

Code :

```
class TreeNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
```

```
root = TreeNode('R')
nodeA = TreeNode('A')
nodeB = TreeNode('B')
nodeC = TreeNode('C')
nodeD = TreeNode('D')
nodeE = TreeNode('E')
nodeF = TreeNode('F')
nodeG = TreeNode('G')
```

```
root.left = nodeA
root.right = nodeB
```

```
nodeA.left = nodeC
nodeA.right = nodeD
```

```
nodeB.left = nodeE
nodeB.right = nodeF
```

```
nodeF.left = nodeG
print("root.right.left.data:", root.right.left.data)
print("Raj_45")
```

```
class TreeNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

root = TreeNode('R')
nodeA = TreeNode('A')
nodeB = TreeNode('B')
nodeC = TreeNode('C')
nodeD = TreeNode('D')
nodeE = TreeNode('E')
nodeF = TreeNode('F')
nodeG = TreeNode('G')

root.left = nodeA
root.right = nodeB

nodeA.left = nodeC
nodeA.right = nodeD

nodeB.left = nodeE
nodeB.right = nodeF

nodeF.left = nodeG
print("root.right.left.data:", root.right.left.data)
print("Raj_45")
```

Output :

```
root.right.left.data: E
Raj_45
```

c. Search for a node in a binary search tree.

Code :

```
class TreeNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

root = TreeNode('R')
nodeA = TreeNode('A')
nodeB = TreeNode('B')
nodeC = TreeNode('C')
nodeD = TreeNode('D')
nodeE = TreeNode('E')
nodeF = TreeNode('F')
nodeG = TreeNode('G')

root.left = nodeA
root.right = nodeB

nodeA.left = nodeC
nodeA.right = nodeD

nodeB.left = nodeE
nodeB.right = nodeF

nodeF.left = nodeG
#Test
print("root.right.left.data:", root.right.left.data)

def search(root, key):
    if root is None:
        return False

    print("Checking:", root.data)

    if root.data == key:
        return True

    return search(root.left, key) or search(root.right, key)
```

```
key = input("Enter element to search: ")
```

```
# Search
```

```
if search(root, key):
```

```
    print("Element Found")
```

```
else:
```

```
    print("Element Not Found")
```

```
print("Raj_45")
```

```
class TreeNode:
```

```
    def __init__(self, data):
```

```
        self.data = data
```

```
        self.left = None
```

```
        self.right = None
```

```
root = TreeNode('R')
```

```
nodeA = TreeNode('A')
```

```
nodeB = TreeNode('B')
```

```
nodeC = TreeNode('C')
```

```
nodeD = TreeNode('D')
```

```
nodeE = TreeNode('E')
```

```
nodeF = TreeNode('F')
```

```
nodeG = TreeNode('G')
```

```
root.left = nodeA
```

```
root.right = nodeB
```

```
nodeA.left = nodeC
```

```
nodeA.right = nodeD
```

```
nodeB.left = nodeE
```

```
nodeB.right = nodeF
```

```
nodeF.left = nodeG
```

```
#Test
```

```
print("root.right.left.data:", root.right.left.data)
```

```

def search(root, key):
    if root is None:
        return False

    print("Checking:", root.data)

    if root.data == key:
        return True

    return search(root.left, key) or search(root.right, key)

key = input("Enter element to search: ")

# Search
if search(root, key):
    print("Element Found")
else:
    print("Element Not Found")
print("Raj_45")

```

Output :

```

C:\Users\11-17\AppData\Local\Programs\
root.right.left.data: E
Enter element to search: C
Checking: R
Checking: A
Checking: C
Element Found
Raj_45
|

```

b. Insert nodes into a binary search tree.

Code :

```
class TreeNode:
```

```
    def __init__(self, data):
```

```
        self.data = data
```

```
        self.left = None
```

```
        self.right = None
```

```
root = TreeNode('R')
```

```
nodeA = TreeNode('A')
```

```
nodeB = TreeNode('B')
```

```
nodeC = TreeNode('C')
```

```
nodeD = TreeNode('D')
```

```
nodeE = TreeNode('E')
```

```
nodeF = TreeNode('F')
```

```
nodeG = TreeNode('G')
```

```
root.left = nodeA
```

```
root.right = nodeB
```

```
nodeA.left = nodeC
```

```
nodeA.right = nodeD
```

```
nodeB.left = nodeE
```

```
nodeB.right = nodeF
```

```
nodeF.left = nodeG
```

```
#Test
```

```
print("root.right.left.data:", root.right.left.data)
```

```
def insert(root, data):
```

```
    if root is None:
```

```
        return TreeNode(data)
```

```
    if data < root.data:
```

```
        root.left = insert(root.left, data)
```

```
    elif data > root.data:
```

```
        root.right = insert(root.right, data)
```

```
return root
```

```
new_node = input("Enter element to insert: ")
```

```
root = insert(root, new_node)
```

```
print("Node inserted successfully.")
```

```
print("Raj_45")
```

```
class TreeNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

root = TreeNode('R')
nodeA = TreeNode('A')
nodeB = TreeNode('B')
nodeC = TreeNode('C')
nodeD = TreeNode('D')
nodeE = TreeNode('E')
nodeF = TreeNode('F')
nodeG = TreeNode('G')

root.left = nodeA
root.right = nodeB

nodeA.left = nodeC
nodeA.right = nodeD

nodeB.left = nodeE
nodeB.right = nodeF

nodeF.left = nodeG
#Test
print("root.right.left.data:", root.right.left.data)
```

```

def insert(root, data):
    if root is None:
        return TreeNode(data)

    if data < root.data:
        root.left = insert(root.left, data)
    elif data > root.data:
        root.right = insert(root.right, data)

    return root

new_node = input("Enter element to insert: ")
root = insert(root, new_node)

print("Node inserted successfully.")
print("Raj_45")

```

Output :

```

>>> RESTART: C:/Users/11-11/AppData
root.right.left.data: E
Enter element to insert: R
Node inserted successfully.
Raj_45
>

```